Welcome to GraphRAG

- 👉 Microsoft Research Blog Post
- 👉 GraphRAG Arxiv

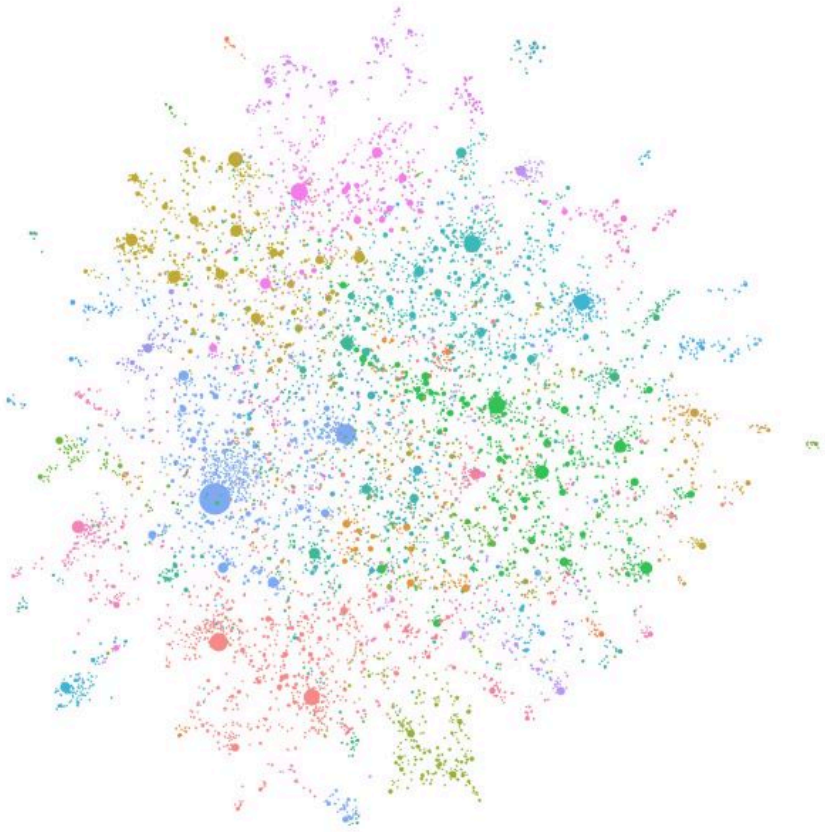

Figure 1: An LLM-generated knowledge graph built using GPT-4 Turbo.

GraphRAG is a structured, hierarchical approach to Retrieval Augmented Generation (RAG), as opposed to naive semantic-search approaches using plain text snippets. The GraphRAG process involves extracting a knowledge graph out of raw text, building a community hierarchy, generating summaries for these communities, and then leveraging these structures when perform RAG-based tasks.

To learn more about GraphRAG and how it can be used to enhance your language model's ability to reason about your private data, please visit the Microsoft Research Blog Post.

Get Started with GraphRAG 🚀

To start using GraphRAG, check out the *Get Started* guide. For a deeper dive into the main sub-systems, please visit the docpages for the Indexer and Query packages.

GraphRAG vs Baseline RAG 🔍

Retrieval-Augmented Generation (RAG) is a technique to improve LLM outputs using real-world information. This technique is an important part of most LLM-based tools and the majority of RAG approaches use vector similarity as the search technique, which we call *Baseline RAG*. GraphRAG uses

knowledge graphs to provide substantial improvements in question-and-answer performance when reasoning about complex information. RAG techniques have shown promise in helping LLMs to reason about *private datasets* - data that the LLM is not trained on and has never seen before, such as an enterprise's proprietary research, business documents, or communications. *Baseline RAG* was created to help solve this problem, but we observe situations where baseline RAG performs very poorly. For example:

- Baseline RAG struggles to connect the dots. This happens when answering a question requires traversing disparate pieces of information through their shared attributes in order to provide new synthesized insights.
- Baseline RAG performs poorly when being asked to holistically understand summarized semantic concepts over large data collections or even singular large documents.

To address this, the tech community is working to develop methods that extend and enhance RAG. Microsoft Research's new approach, GraphRAG, creates a knowledge graph based on an input corpus. This graph, along with community summaries and graph machine learning outputs, are used to augment prompts at query time. GraphRAG shows substantial improvement in answering the two classes of questions described above, demonstrating intelligence or mastery that outperforms other approaches previously applied to private datasets.

The GraphRAG Process 🤖

GraphRAG builds upon our prior research and tooling using graph machine learning. The basic steps of the GraphRAG process are as follows:

Index

- Slice up an input corpus into a series of TextUnits, which act as analyzable units for the rest of the process, and provide fine-grained references in our outputs.
- Extract all entities, relationships, and key claims from the TextUnits.
- Perform a hierarchical clustering of the graph using the Leiden technique. To see this visually, check out Figure 1 above. Each circle is an entity (e.g., a person, place, or organization), with the size representing the degree of the entity, and the color representing its community.
- Generate summaries of each community and its constituents from the bottom-up. This aids in holistic understanding of the dataset.

Query

At query time, these structures are used to provide materials for the LLM context window when answering a question. The primary query modes are:

- *Global Search* for reasoning about holistic questions about the corpus by leveraging the community summaries.
- *Local Search* for reasoning about specific entities by fanning-out to their neighbors and associated concepts.
- *DRIFT Search* for reasoning about specific entities by fanning-out to their neighbors and associated concepts, but with the added context of community information.
- *Basic Search* for those times when your query is best answered by baseline RAG (standard top *k* vector search).

Prompt Tuning

Using *GraphRAG* with your data out of the box may not yield the best possible results. We strongly recommend to fine-tune your prompts following the Prompt Tuning Guide in our documentation.

Versioning

Please see the breaking changes document for notes on our approach to versioning the project.

Always run `graphrag init --root [path] --force` *between minor version bumps to ensure you have the latest config format. Run the provided migration notebook between major version bumps if you want to avoid re-indexing prior datasets. Note that this will overwrite your configuration and prompts, so backup if necessary.*